Autonomous MirAI Student Policy Advisor

Course: Generative AI & LangChain Architecture
Format: Full-Stack Solo Project
Required Dataset: `Mirai_SoT_Policy_Handbook_2026.pdf`

1. Project Overview

The university administration handles a high volume of repetitive inquiries regarding attendance grading, medical leave procedures, and club operations. The objective of this capstone is to develop a production-grade Retrieval-Augmented Generation (RAG) system to automate these responses.

You will build an independent AI assistant that ingests the official MirAI student handbook and acts as an autonomous policy advisor. The system must operate under strict constraints: it is absolutely critical that the assistant does not hallucinate or invent institutional policies. All answers must be strictly confined to the facts provided in the source document.

2. Technical Requirements

2.1 Data Pipeline (Ingestion & Vectorization)

- **Document Loader:** Utilize `PyPDFLoader` to ingest the provided `Mirai_SoT_Policy_Handbook_2026.pdf`.
- **Semantic Chunking:** Process the text using `RecursiveCharacterTextSplitter`. You must configure an appropriate `chunk_size` (e.g., 1000) and `chunk_overlap` (e.g., 200) to preserve semantic context across policy clauses.
- **Vector Database:** Embed the text chunks using `GoogleGenerativeAIEmbeddings` (model: `text-embedding-004`) and store them in a local ChromaDB instance.

2.2 Retrieval Architecture (LangChain & LCEL)

- **Advanced Retrieval:** Standard vector search is insufficient for this use case due to the vocabulary gap between casual student queries and formal handbook terminology. You must implement a `MultiQueryRetriever` to expand and rewrite user queries prior to database execution.
- **Generator Configuration:** Use `ChatGoogleGenerativeAI` (`gemini-3.8-flash`). The model temperature must be set to `0.0` to ensure deterministic, factual responses.
- **Guardrails:** Your System Prompt must explicitly restrict the LLM from relying on its base training data. If the answer to a user's query cannot be found within the retrieved context, the system must politely decline to answer.

2.3 Application Layer (FastAPI & Streamlit)

- **Backend Interface (FastAPI):** Develop and expose two asynchronous REST endpoints:
    - `POST /ingest`: Accepts a PDF file via `multipart/form-data`, processes the chunks, and updates the ChromaDB vector store.
    - `POST /chat`: Accepts a JSON payload containing the user's question, executes the LCEL RAG chain, and returns the generated answer.
- **Frontend Interface (Streamlit):** Develop a chat interface designed for student use. The frontend must implement error handling to manage server timeouts or backend connection failures gracefully.

3. Certification Audit (Test Suite)

Prior to submission, you are required to validate your RAG pipeline against the following test queries. Your system must retrieve and synthesize the correct information strictly from the handbook.

1. **Precision Verification:** *"I have 72% attendance. How many attendance marks will I get?"*
    - *Expected Outcome:* The system must state the student will receive 4 marks.
2. **Multi-Hop Reasoning:** *"I study at the Ratnam campus. I got sick and need medical leave. Who do I email and how many days do I have to submit my documents?"*
    - *Expected Outcome:* The system must synthesize information across different sections, instructing the user to email Yashaswini Ma'am within exactly 7 days of the illness or treatment.
3. **Process Verification:** *"We want to start a new Cybersecurity society under the Tech Club. Do we ask Management directly?"*
    - *Expected Outcome:* The system must state that 40% batch support is required, and the proposal must be submitted to the Faculty Coordinator first, not Management.
4. **Negative Constraint Testing:** *"How much is the fine for smoking a cigarette on campus?"*
    - *Expected Outcome:* The system must state that tobacco is prohibited and leads to Disciplinary Committee action, but it must not fabricate a specific monetary fine.

4. Automated Evaluation Deliverable

Manual quality assurance is not scalable. You must implement the LLM-as-a-judge pipeline covered in class to benchmark your system's accuracy.

1. Execute the 4 test queries listed in Section 3 through your RAG system.

2. Pass the context, question, and generated answers through your automated evaluation script.
3. Log the grading output (a Score from 1-5 and the Reasoning) into a local file named `rag_eval_scores.csv`.

5. Submission Guidelines

Push the following files to a public GitHub repository and submit the link via the learning management system:

- `backend.py` (FastAPI application)
- `frontend.py` (Streamlit application)
- `rag_eval_scores.csv` (Automated benchmarking logs)
- `requirements.txt`
- `README.md` (Must contain standard operating procedures for executing both the backend and frontend servers locally)